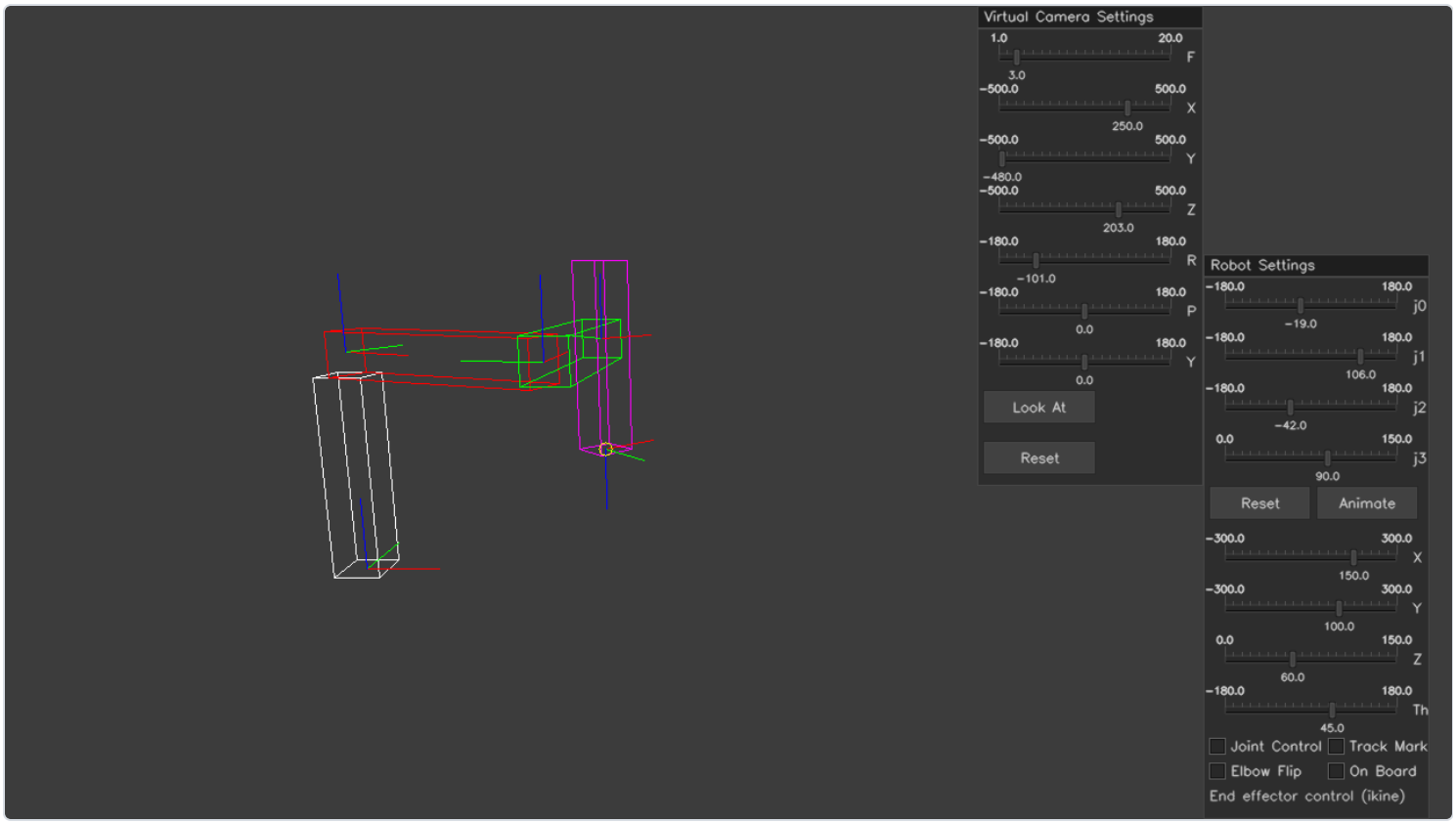
Lab 6 – Inverse Kinematics (SCARA): Summary of Changes

Branch: **Lab06_A_Experiment** · ELEX 7825 · Builds in Visual Studio 18, Debug and Release x64, no warnings. Changes are uncommitted in the working tree.

1. What was done, in one paragraph

Menu option **(6) Lab 6** now runs the Lab 5 SCARA robot with inverse kinematics added. A new `ikine()` turns a target end effector pose (X, Y, Z, θ) into the four joint values, which is the reverse of Lab 5's `fkin()`. The Robot Settings window gained a **Joint Control** checkbox: ticked, the joint sliders drive the robot as in Lab 5; unticked, the X Y Z Th sliders drive it through `ikine()`. The Animate button runs the Lab 5 joint animation in joint control and a new straight-line end effector animation otherwise. For Part B, **On Board** draws the robot on the ChArUco board and **Track Mark** finds ArUco marker ID 50, draws a box around it, and locks the end effector onto it. Lab 3, 4 and 5 code is untouched: Lab 6 is new functions that call the old ones.



End effector control. The X Y Z Th sliders are the input (150, 100, 60 mm, 45°); ikine() worked out j0 = −19°, j1 = 106°, j2 = −42°, j3 = 90 mm and wrote them onto the joint sliders. The yellow circle marks the target.

2. How to try it (Part A needs no webcam)

- Build and run, type **6** at the menu. It starts on the virtual camera in joint control, which behaves like Lab 5.
- Untick **Joint Control**. Now drag **X, Y, Z, Th**: the arm follows and the j0–j3 sliders show the answer from `ikine()`.
- Tick **Elbow Flip** to see the other solution for the same target (the elbow bends the other way, the tool does not move).
- Drag X and Y both to the far right. The status line turns red, "Target unreachable", and the arm stays where it last was.
- Press **Animate** with Joint Control unticked: the end effector follows four straight lines and leaves a cyan trail. With Joint Control ticked, Animate runs the Lab 5 joint sequence.
- **Reset** returns to home and to joint control.
- Part B (needs webcam, board and marker 50): tick **On Board**, then **Track Mark**.

3. Files changed

File	Change
Robot.h	New "LAB 6" block of private members and helper functions; the commented-out `ikine()` stub is replaced by the real declaration plus `create_scara_lab6()` and `draw_scara_lab6()`.
Robot.cpp	New "LAB6 - SCARA robot inverse kinematics" section appended at the end. Nothing above it was edited.
CameraReal.h / .cpp	New `detect_single_marker()` and two getters for the marker's rvec / tvec. Existing functions untouched.
7825_Template.cpp	`lab6()` filled in, same shape as `lab5()`.
docs/	This summary (HTML, PDF, Word), screenshots in docs/images, and Lab7_Discussion.md.

4. Where each lab requirement lives

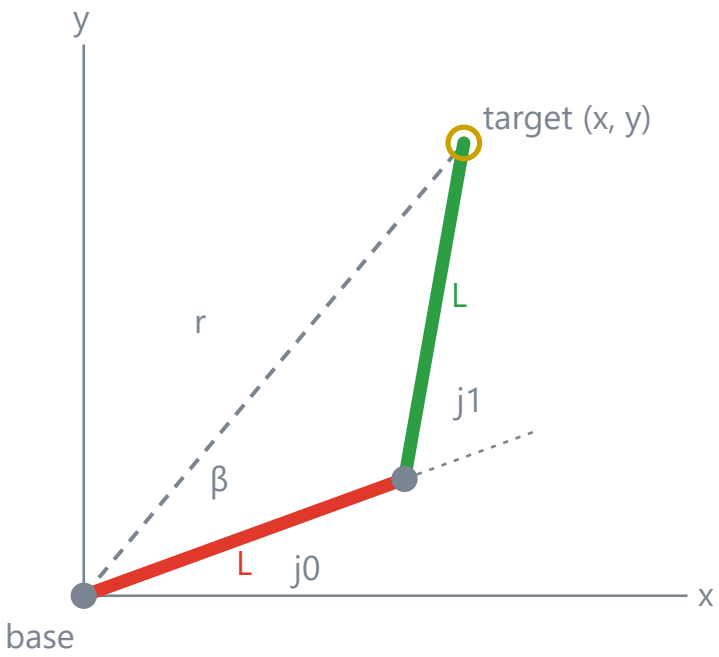
Lab item	Requirement	Code
Part A-A	Closed-form inverse kinematic equations	Section 5 of this document; comments at the top of `ikine()`
Part A-B	`ikine` method: pose in, joints out	`CRobot::ikine()`
Part A-C	Checkbox for joint / end effector control; show joints on sliders and draw the arm	`update_scara_settings_lab6()`, `draw_scara_lab6()`, `set_joint_sliders()`
Part A-D	Animate moves the end effector along a linear path	`animate_scara_ikine()`, `IK_WAYPOINTS`
Part B-A	SCARA on the board, joint animation still works	`draw_scara_lab6()` steps 1–4 (same as Lab 5), `animate_scara()` reused
Part B-B	Checkbox, marker ID 50 pose (X,Y,Z,R,P,Y), Rodrigues, 3D box	`CCameraReal::detect_single_marker()`, `createHT_rvec()`, `track_marker()`, marker box in `draw_scara_lab6()`
Part B-C	Marker pose → pose sliders → ikine	End of `track_marker()`, then the end effector branch of `update_scara_settings_lab6()`

5. Part A-A: the inverse kinematics, derived

Forward kinematics answers "the joints are here, where is the tool?". Inverse kinematics answers "I want the tool there, what should the joints be?". Start from what `fkin()` computes. Multiplying out the Lab 5 transform chain, with L = 0.15 m (link length) and H = 0.15 m (height of joint 1), gives four plain equations:

```
x     = L cos(j0) + L cos(j0 + j1)
y     = L sin(j0) + L sin(j0 + j1)
z     = H - j3
theta = j0 + j1 + j2
```

Seen from above, the SCARA is a 2-link planar arm: j0 and j1 alone decide X and Y. j3 only affects Z, and j2 only affects θ. That is why the problem splits into four small steps.



Top view of the SCARA as a 2-link arm. Red = link 1, green = link 2. The dashed line from the base to the target has length r, where $r^2 = x^2 + y^2$. β is the angle between link 1 and that line.

Step 1: elbow angle j1 (law of cosines)

Square the x and y equations and add them. The j0 terms cancel and only j1 is left:

```
x² + y² = 2L² + 2L² cos(j1)

cos(j1) = (x² + y² - 2L²) / (2L²)
j1      = ± acos( cos(j1) )
```

Two things fall out of this one line. If the right-hand side is bigger than 1, the target is farther than 2L = 300 mm and **there is no solution**. Otherwise there are **two solutions**, +j1 and −j1: the same hand position with the elbow on either side. The Elbow Flip checkbox picks between them.

Step 2: shoulder angle j0

The direction from the base to the target is `atan2(y, x)`. Link 2 bends the tip away from the line of link 1 by an angle β, so link 1 must point at the target direction minus β:

```
β  = atan2( L sin(j1), L + L cos(j1) )
j0 = atan2(y, x) - β
```

`atan2` is used instead of `atan` because it gives the correct quadrant for all signs of x and y.

Step 3: prismatic joint j3

```
j3 = H - z
```

Valid only if 0 ≤ j3 ≤ 150 mm, the travel of the quill.

Step 4: wrist angle j2

As the lab suggests, `fkin()` is reused: call it with the j0 and j1 just found and j2 = 0 to see which way the tool would point, then turn the wrist by the difference.

```
j2 = theta_target - fkin(j0, j1, 0, j3).theta      (= theta - j0 - j1)
```

All angles are wrapped back into −180…180° to match the sliders.

6. Part A-B: `ikine()` in code

Units match `fkin()` exactly (metres and degrees), so the two functions are inverses of each other: `fkin(ikine(pose)) == pose`.

```
bool CRobot::ikine(Vec4d pose, Vec4d& joints, bool elbow_flip)

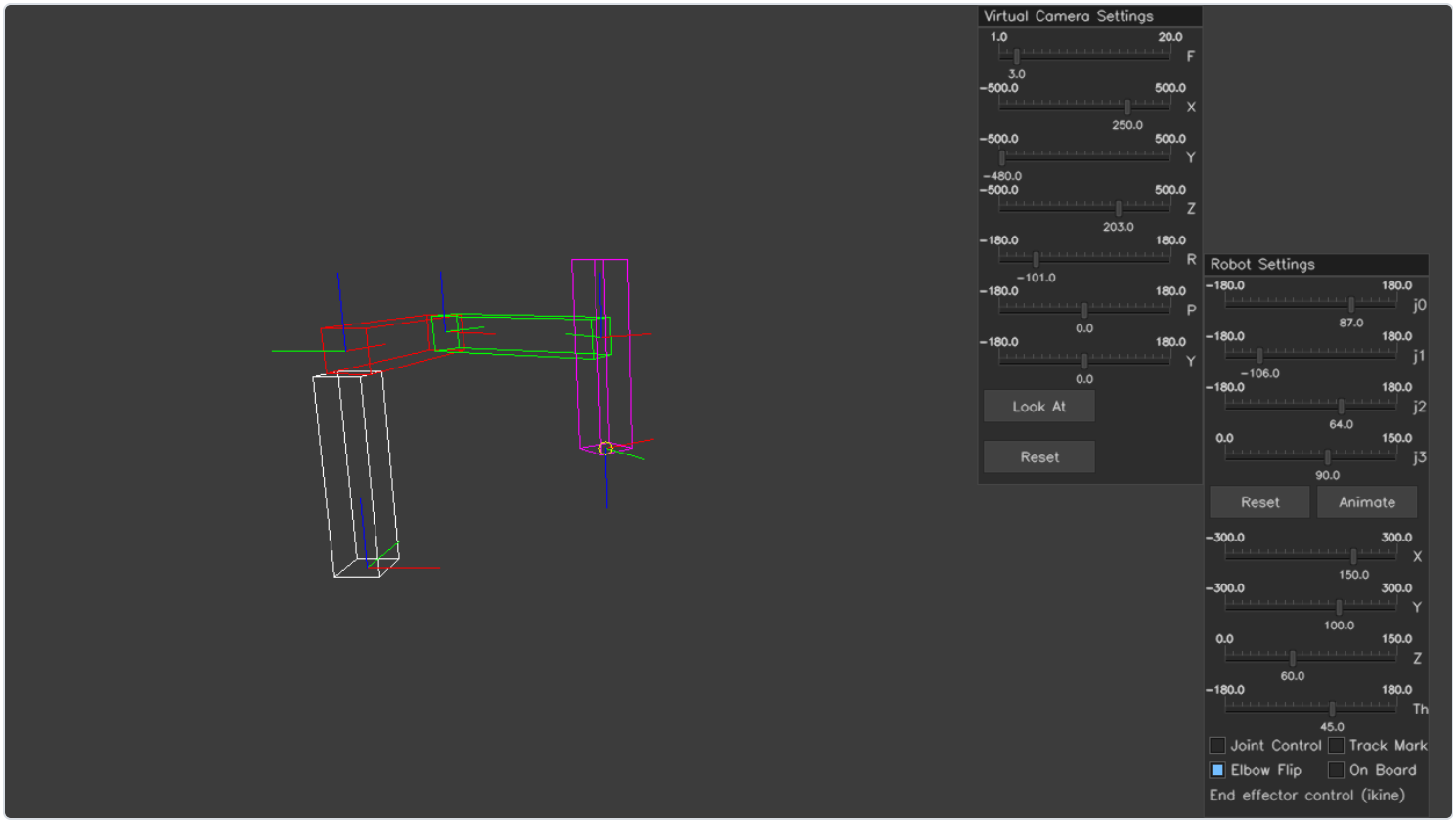
// j1: law of cosines
double c1 = (x * x + y * y - 2 * L * L) / (2 * L * L);
if (c1 > 1.000001 || c1 < -1.000001)
    return false;                       // out of reach
double j1 = acos(c1);
if (elbow_flip) j1 = -j1;

// j0: direction to target minus the bend
double j0 = atan2(y, x) - atan2(L * sin(j1), L + L * cos(j1));

// j3: from the height
double j3 = SCARA_JOINT1_HEIGHT - z;   // rejected if outside 0 .. 150 mm

// j2: fkin with j2 = 0, then the remainder
Vec4d reached = fkin(Vec4d(j0, j1, 0.0, j3));
double j2 = wrap_180(theta - reached[3]);
```

It returns `bool` (the stub in Robot.h hinted at that): `true` and fills `joints`, or `false` and leaves `joints` alone. The tiny 0.000001 tolerance exists because the home pose is exactly at full stretch, where float rounding can push cos(j1) to 1.0000001.



Same target as the first screenshot with Elbow Flip ticked: j1 changed sign (106° → −106°), j0 and j2 changed to compensate, and the tool is in the same place.

7. Part A-C: how the GUI ties fkin and ikine together

`draw_scara_lab6()` draws the robot, then calls `update_scara_settings_lab6()`, which works in one of two directions each frame:

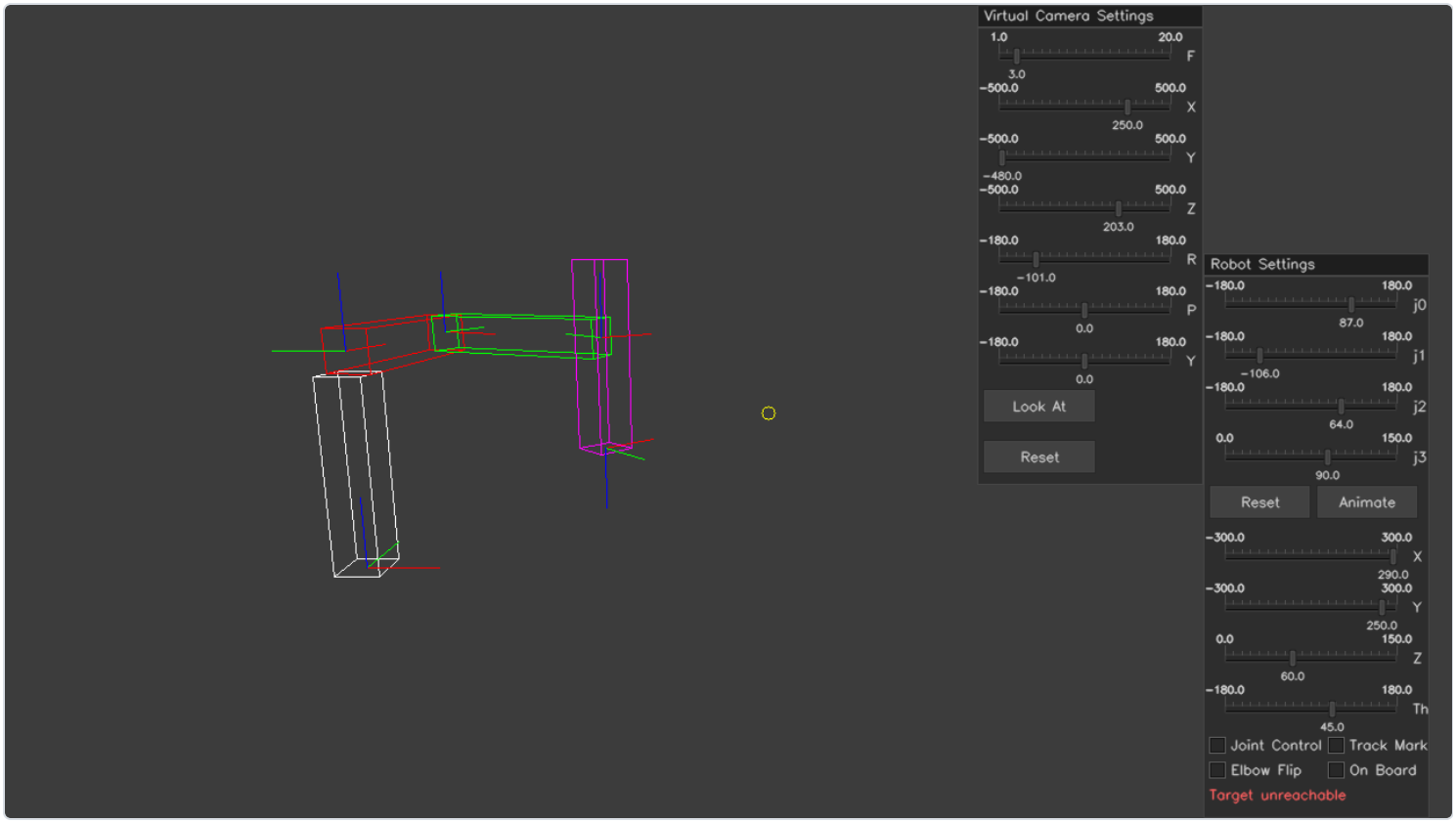
	Joint Control ticked	**Joint Control unticked**
Input	j0–j3 sliders (`_joint[]`)	X Y Z Th sliders (`_pose[]`)
Calculation	`fkin(get_joints())`	`ikine(pose, joints, _elbow_flip)`
Output shown on	X Y Z Th sliders	j0–j3 sliders, via `set_joint_sliders()`
Animate runs	`animate_scara()` (Lab 5)	`animate_scara_ikine()` (Lab 6)

Either way the result ends up in `_joints_lab6`, and that is what the robot is drawn from, using Lab 5's `scara_transforms()`, `transformPoints()`, `drawBox()` and `drawCoord()`.

Why a separate `_joints_lab6`? The Lab 5 joint sliders are whole numbers (degrees, mm). Rounding an ikine answer to whole degrees would move the tool by up to about 2.5 mm at

full reach, and the straight-line animation would wobble. So the exact answer is kept as doubles for drawing, and the int sliders only display it.

Unreachable targets: `_ik_ok` goes false, `_joints_lab6` is not updated (the arm holds its last good pose), and the status line shows "Target unreachable" in red. The yellow circle still shows where the target is.



Target at X = 290, Y = 250 mm is 383 mm from the base, beyond the 300 mm reach. The arm holds its last pose and the yellow target circle sits out on its own.

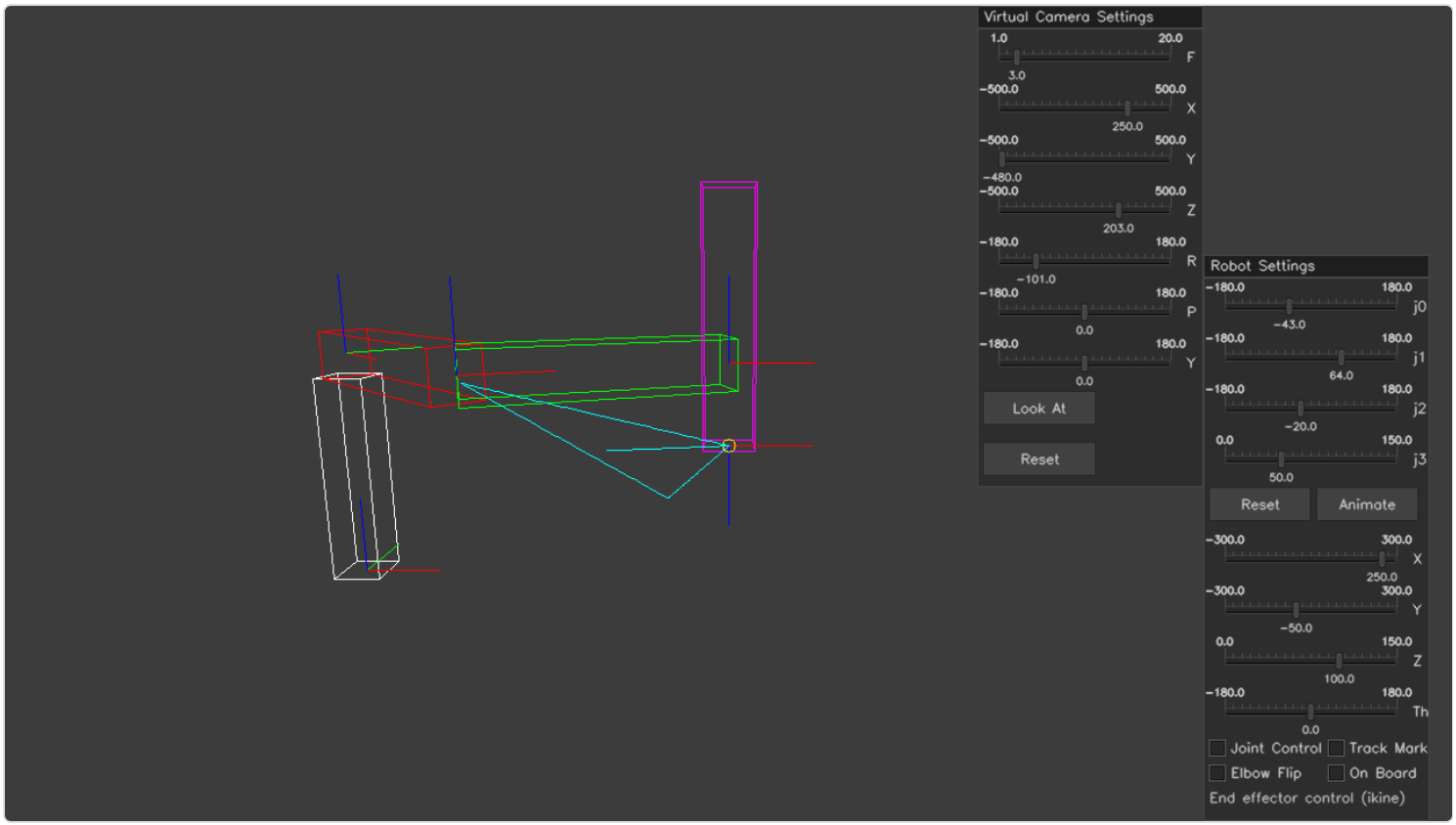
Switching modes smoothly: while in joint control, the pose sliders are continuously filled from fkin and Elbow Flip is set to match the sign of j1, so unticking Joint Control does not make the arm jump.

8. Part A-D: straight-line animation

`animate_scara_ikine()` uses the same state-machine idea as Lab 5's `animate_scara()`: `_do_animate` is which move is running and `_anim_count` is the step within it. The difference is *what* gets stepped. Lab 5 stepped the joints. Lab 6 steps the *target pose* in a straight line and lets ikine find the joints every frame:

```
t    = _anim_count / IK_ANIM_STEPS            // 0 → 1 over 60 frames
pose = _anim_start + (target - _anim_start) * t   // linear interpolation
```

There are four moves through the `IK_WAYPOINTS` table (approach, straight across at constant height, out-and-down while turning the tool 90°, and back). The joints move in curves so that the tool moves in a line, which is the whole point of inverse kinematics. A cyan trail is drawn from `fkin()` of each ikine answer, so the trail shows where the tool really went, not just where it was asked to go.



End of the ikine animation. The cyan trail is the path of the end effector: four straight segments.

9. Part B: tracking marker ID 50

Part B-A: robot on the board

The On Board checkbox does what Lab 5's "Draw on ArUco board" did, with the same steps and the same board alignment `createHT((0,0,0), (180,0,90))`. In joint control the Animate button still runs the Lab 5 joint animation.

Part B-B: marker pose

About `estimatePoseSingleMarkers`: the lab names this function, but it is not in the OpenCV 4.10 build supplied for the course (it lives in the separate contrib module since 4.7). `CCameraReal::detect_single_marker()` does the same thing the same way: `ArucoDetector::detectMarkers` finds the four corners of marker 50, then `solvePnP` solves for the pose given the marker's known size. This is the approach already used for the board in Lab 4.

solvePnP returns `rvec` and `tvec`. `rvec` is an axis-angle vector, not a rotation matrix: its direction is the rotation axis and its length is the angle. `createHT_rvec()` calls `Rodrigues()` to convert it into a 3×3 rotation matrix and packs it with `tvec` into the usual 4×4 transform.

The camera reports both poses relative to itself. To get the marker relative to the robot, chain the transforms (read right to left, like Lab 2):

```
cTb = board  in the camera frame      (from detect_markers, Lab 4)
cTm = marker in the camera frame      (from detect_single_marker)
bTw = robot world in the board frame  (the board alignment, called "base")

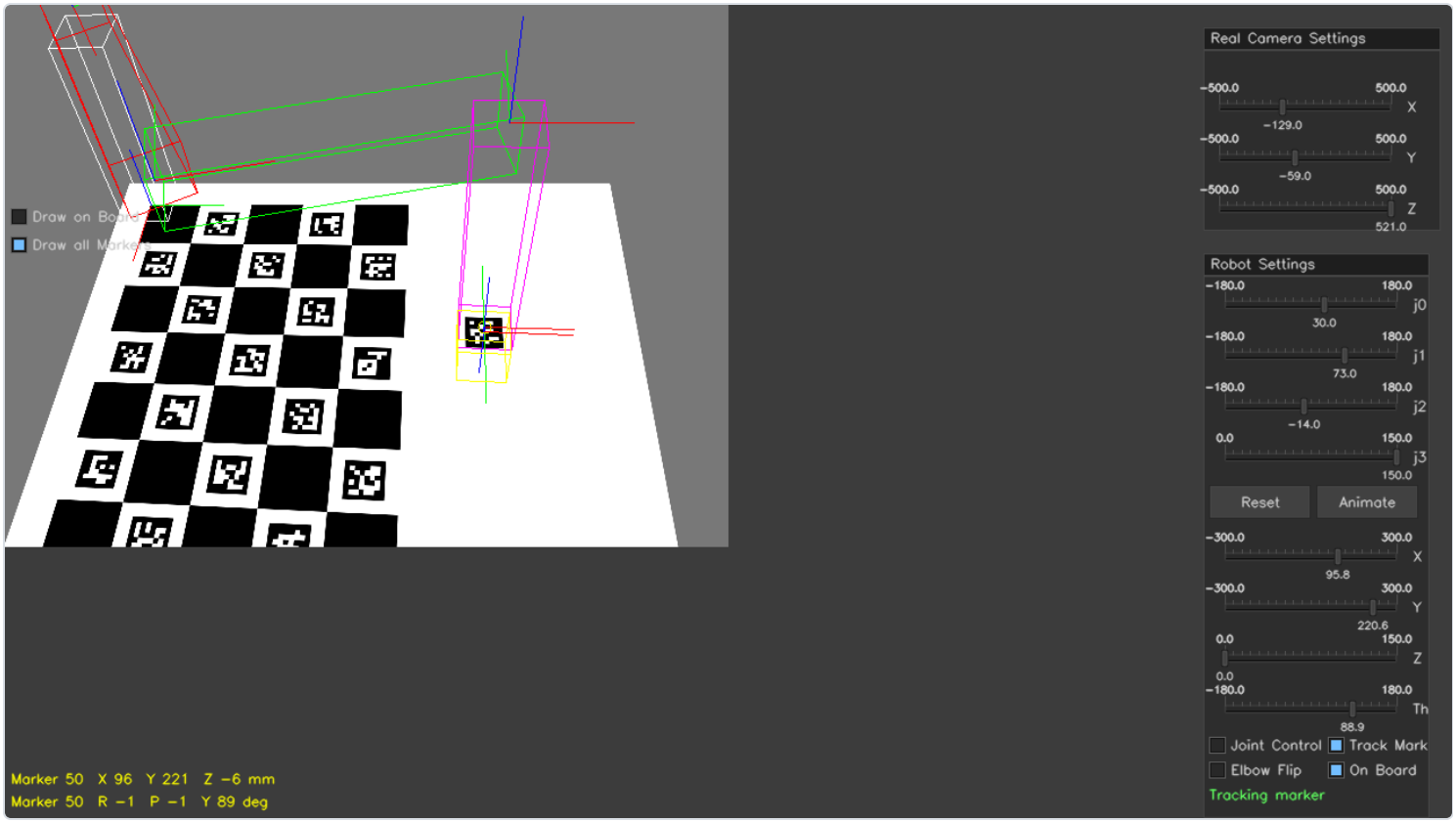
bTm = inv(cTb) * cTm       marker in the board frame  → used to draw the yellow box
wTm = inv(bTw) * bTm       marker in the robot frame  → used for ikine
```

X, Y, Z are the last column of `wTm`. Roll, pitch, yaw are pulled out of its rotation part using the same Rz·Ry·Rx order that `createHT()` builds, so they mean the same thing as everywhere else in the program. They are printed at the bottom left.

The yellow box is `createBox(30 mm)` moved by `bTm` and shifted half a cube down, because the marker is printed on the top face of the paper cube.

Part B-C: locking the robot on

`track_marker()` ends by copying the marker's X, Y, Z and yaw onto the `_pose[]` sliders (clamped to the slider ranges). Ticking Track Mark also forces On Board on and Joint Control off, so `update_scara_settings_lab6()` runs ikine on those sliders exactly as if you had dragged them. No special tracking path was needed: the marker is just another hand on the sliders.



Part B checked with a SYNTHETIC camera image (a rendered board with marker 50 lying flat beside it), not a real webcam. The robot stands on the board origin, the yellow box is on the marker, and the quill has come down onto it.

10. How it was tested

A separate test program (kept outside the project, nothing added to the repo) compiled the real project sources and drove them without the mouse. Results:

Check	Result
20,000 random joint sets → fkin → ikine → fkin	0 failures. Worst position error 0.00005 mm, worst angle error 0.00002°
Both elbow solutions for (150, 100, 60 mm, 45°)	j1 = ±106.1°, both give exactly the same fkin pose
Unreachable: 400 mm away, Z = 200 mm, Z = −10 mm	All return false, output left untouched
Edge cases: home (full stretch), target over the base (x = y = 0)	Both solve
Lab 5 joint animation inside Lab 6	Runs to completion, joints return to home
ikine animation, 240 frames	End effector never more than 0.00003 mm off the straight path
Unreachable target in the GUI	Arm holds its pose, warning shown
Part B with a synthetic image, marker truly at (100, 220, 0) mm, yaw 90°	Board found; marker measured at (96, 221, −6) mm, R −1°, P −1°, Y 89°; robot locked on

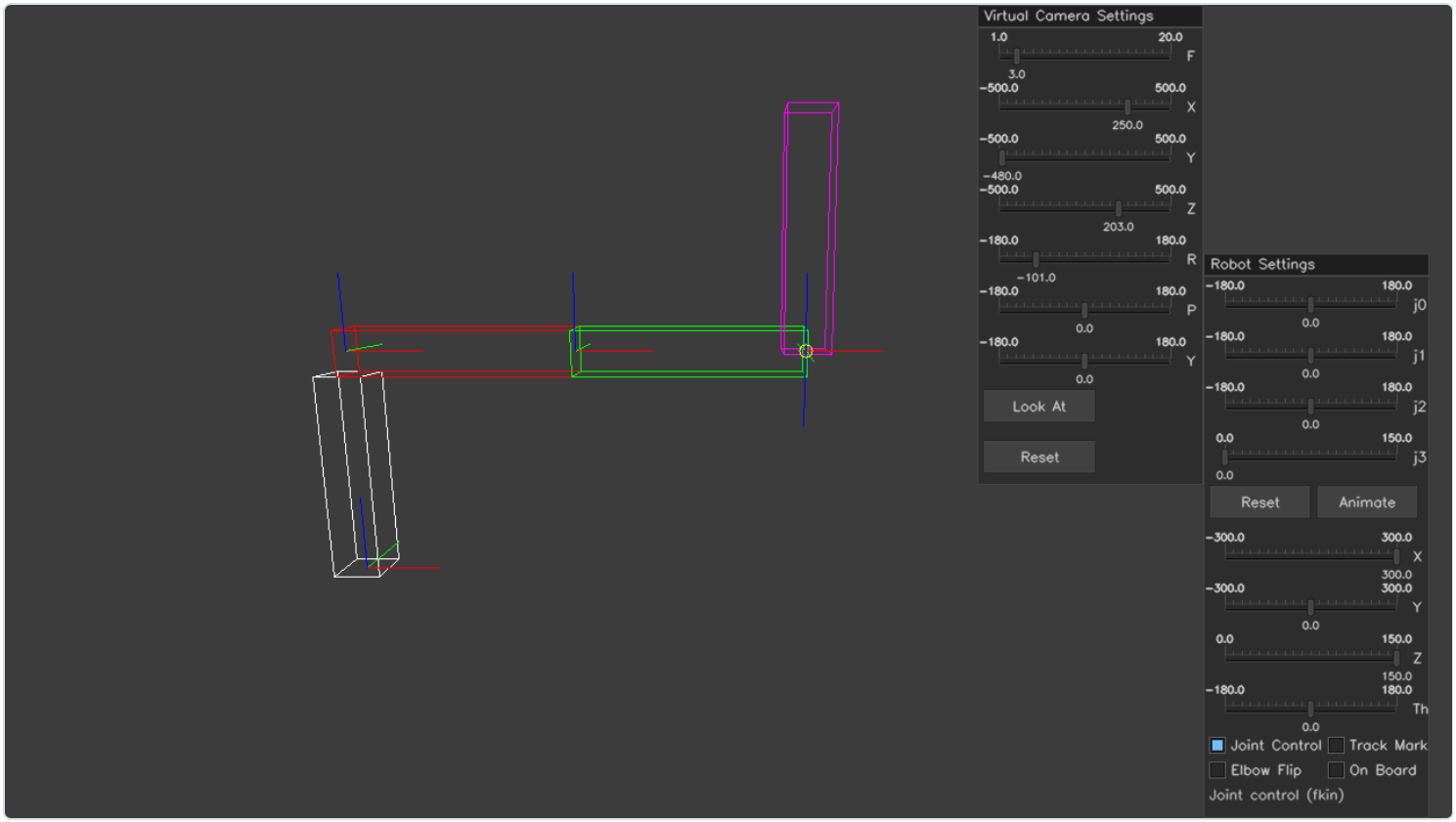
Not tested: a real webcam, a real printed board, a real marker cube. The synthetic test confirms the maths and the frame directions, but not lighting, focus, or your printed sizes. The few-mm error in the last row is normal for a 21 mm marker seen from half a metre.

11. Things to check when you have the board

- **Marker size.** `MARKER_SIZE = 0.021f` and `MARKER_BOX_SIZE = 0.03f` near the top of the Lab 6 section were measured off the PDF page, not a printout. Measure the black square with a ruler. If the size is wrong, the marker will appear too near or too far.
- **Marker ID and dictionary.** The code assumes the cube markers are from `DICT_6X6_250`, the same as the board. If "Marker 50 not found" stays up with the cube in view, this is the first thing to check.
- **Reach.** The robot stands on the board origin and reaches 300 mm. Beyond that you get "Target unreachable".
- **Height.** The target is the marker itself (top of the cube, about 30 mm up). Pose noise in Z is a few mm; Z is clamped at 0 so a marker lying flat still tracks.

12. Function reference (all new in Lab 6)

Function	Part	What it does	Builds on
ikine()	A-B	Pose → joints, false if unreachable	fkin()
create_scara_lab6()	A	Builds the robot and sets Lab 6 defaults	create_scara(), init_lab6()
init_lab6()	A-C	Home pose, joint control, flags cleared	init(), get_joints(), fkin()
set_joint_sliders()	A-C	(deg, deg, deg, m) → int sliders	opposite of get_joints()
update_scara_settings_lab6()	A-C, A-D, B	Sliders, buttons, checkboxes; runs fkin or ikine	animate_scara(), fkin(), ikine()
animate_scara_ikine()	A-D	Steps the target pose along straight lines	same pattern as animate_scara()
draw_scara_lab6()	A, B	Draws robot, target, trail, marker box, UI	scara_transforms(), createBox(), transformPoints(), drawBox(), drawCoord()
to_image()	–	One 3D point → pixel with the active camera	both cameras' transform_to_image()
createHT_rvec()	B-B	rvec, tvec → 4×4 transform	Rodrigues()
track_marker()	B-B, B-C	Marker pose in robot frame → pose sliders	detect_single_marker(), createHT_rvec()
CCameraReal::detect_single_marker()	B-B	Finds one marker, solves its pose	same approach as detect_markers()



Starting view of Lab 6: joint control at home, the same as Lab 5 with the new checkboxes at the bottom of Robot Settings.